



JS

JS Aula 01.2:

Funções

Funções

Uma função em JavaScript é um conjunto de códigos (*que podemos chamar de “linhas” ou “bloco”*) que só serão executados quando a função em questão for solicitada.

Essa solicitação pode ser feita, por exemplo, através de um **evento**.

function {

bloco

}

{ }

JS

Nome da função

Apesar do fato de no JavaScript existir as funções anônimas (*que não possuem nome*), para a função funcionar corretamente é necessário atribuir um nome para ela. Geralmente o **nome da função** está ligada a **ação que ela irá executar**.

function **ação** () {

bloco

}

{ }

JS

Parâmetros da função

Além do nome da função, opcionalmente é possível atribuir parâmetros a essa função. Um parâmetro pode ser, por exemplo, um valor passado para a função que será usado por ela.

function ação (*param*) {

bloco

}

}

JS

Funções anônimas



Em JavaScript, quando você atribui uma função a uma variável, você está criando uma variável que armazena uma referência para essa função. Isso é conhecido como uma "função anônima" ou "função sem nome". Aqui está um exemplo:

// Atribuindo uma função a uma variável

```
var minhaFuncao = function() {  
    console.log("Esta é uma função!")  
}
```

// Chamando a função através da variável

```
minhaFuncao()
```

JS

Arrow functions



A partir do ECMAScript 6 (ES6) e versões mais recentes do JavaScript, você pode usar a sintaxe de **Arrow Function** para criar funções anônimas de forma mais concisa. Aqui está um exemplo:

```
// Atribuindo uma função de seta a uma variável
```

```
var minhaFuncao = () => {  
    console.log("Esta é uma função de seta!")  
}
```

```
// Chamando a função através da variável
```

```
minhaFuncao()
```

JS

IIFE – Funções imediatas



O IIFE significa “Immediately-invoked function expression”, mas podemos chamá-lo de função imediata. Como o próprio nome diz, ela executa a função imediatamente depois de criada. Aqui está um exemplo:

```
const iife = (function() {  
    console.log('Executando imediatamente!')  
})();
```

Nesse caso os parênteses no final da sintaxe é a invocação da função, nesse caso a função será chamada imediatamente ao executar o código.

JS

Comando typeof



O **typeof** é uma palavra-chave em JavaScript que retornará o **tipo da variável** quando você a chama.

O operador **typeof** é útil porque é uma maneira fácil de verificar o tipo de uma variável em seu código.

JS

Comando typeof



```
> var n = 300
```

```
undefined
```

```
> typeof n
```

```
'number'
```

```
> var nome = "José"
```

```
undefined
```

```
> typeof nome
```

```
'string'
```

JS

Concatenação



Para juntarmos (*concatenar*) o conteúdo de um texto ao valor de uma variável devemos usar o sinal de concatenação, que no caso do JavaScript é o **+**

Exemplo:

```
<script>
  // Usuário preenche o prompt com o seu nome esse será gravado na variável nome
  var nome = window.prompt('Qual é seu nome?')
  // Uma caixa de alerta aparece com a mensagem e o nome digitado pelo usuário
  window.alert('É um grande prazer em te conhecer, ' + nome + '!')
</script>
```

JS

Template Strings



A versão mais atual do JavaScript permite também concatenar strings através do **Template String**. Nesse caso fazemos uso de um **placeholder**, que é representado pelos símbolos **\$ { }**. Importante notar que agora **não** usamos **aspas simples** (') na mensagem e sim **crase** (`) o que delimita o nosso **Template String**.

Exemplo:

```
// Usuário preenche o prompt com o seu nome, esse será gravado na variável nome
var nome = window.prompt('Qual é seu nome?')
// Uma caixa de alerta aparece com a mensagem e o nome digitado pelo usuário
window.alert(`É um grande prazer em te conhecer, ${nome}`)
```

JS

Comentários em JavaScript



```
// Duas barras adiciona um comentário em linha única

/* Barra asterísco
   adiciona comentários
   em bloco */
```

JS

Operadores aritméticos em JavaScript



Operador	Descrição	Exemplo	Resultado
+	Adição	5 + 2	7
-	Subtração	5 - 2	3
*	Multiplicação	5 * 2	10
/	Divisão	5 / 2	2.5
%	Módulo (Resto)	5 % 2	1
**	Exponenciação	5 ** 2	25

JS



JS

JS Aula 01.2:

Funções